

PriceWatch - API Express

Explicacao completa da API, rotas, services, DTOs, Prisma, MySQL e integracao com o app

Material de estudo para apresentacao: explica a arquitetura, os fluxos, os arquivos principais, as escolhas tecnicas e o papel de cada parte do codigo.

Projeto: PriceWatch - comparador e monitorador de precos

1. Visao geral da API

A API e o backend do PriceWatch. Ela centraliza autenticao, produtos, importacao externa, alertas, notificacoes e historico de precos. O app Flutter nao acessa o banco diretamente; ele conversa com esta API por HTTP.

Flutter App
-> HTTP JSON
-> Express Routes
-> Controllers
-> Services
-> Prisma Client
-> MySQL

A escolha de Express com TypeScript ajuda a manter rotas simples, tipagem nos DTOs e uma separacao clara entre entrada HTTP e regra de negocio. Prisma foi usado para mapear tabelas MySQL para modelos TypeScript.

2. package.json e dependencias

Dependencia	Papel
express	Cria servidor HTTP e define rotas.
typescript	Permite tipagem estatica no backend.
prisma e @prisma/client	Mapeiam o banco MySQL e executam queries.
bcryptjs	Gera hash de senha e compara senha no login.
jsonwebtoken	Gera e verifica JWT usado em rotas autenticadas.
class-validator	Valida DTOs com decorators como IsString e IsPositive.
class-transformer	Converte string de query/params para number/boolean.
cors	Libera chamadas do app Flutter para a API.
dotenv	Carrega variaveis do arquivo .env.

Scripts importantes: npm run dev sobe a API com ts-node-dev; npm run build compila TypeScript para JavaScript; npm start executa a versao compilada em dist/server.js.

3. app.ts e server.ts - entrada do backend

```
const app = express();
app.use(corsConfig);
app.use(express.json());
app.use(router);
app.use(errorHandler);
```

- reflect-metadata e necessario para decorators usados por class-validator/class-transformer.
- express() cria a aplicacao.
- corsConfig libera a API para ser consumida pelo app.
- express.json() permite receber body JSON.
- router registra todas as rotas do projeto.
- errorHandler fica por ultimo para capturar erros lancados nas camadas anteriores.
- server.ts chama app.listen(env.PORT) e inicia a porta configurada no .env.

4. Configuracao e ambiente

Arquivo	Explicacao
src/config/env.ts	Carrega .env, define PORT, DATABASE_URL, JWT_SECRET e configuracoes do provedor externo.
src/config/cors.ts	Define origin '*' e metodos GET, POST, PUT e DELETE.

Arquivo	Explicacao
src/prisma/client.ts	Cria uma instancia unica do PrismaClient para services usarem.
.env	Guarda DATABASE_URL do MySQL, PORT e JWT_SECRET. Nao deve ser enviado publicamente.

Observacao para apresentar: o projeto esta usando MySQL via Workbench/local, nao Docker. O Prisma usa DATABASE_URL para saber onde esta o banco.

5. Banco de dados - Prisma schema

Model	Campos	Relacoes e sentido no sistema
User	id, name, email, passwordHash, createdAt	Um usuario possui varios alertas. Email e unico. Senha nao e salva pura, so hash.
Product	id, name, store, category, currentPrice, previousPrice, emoji, externalId, updatedAt	Representa uma oferta/produto salvo no banco. Pode ter alertas e historico de preco.
Alert	id, userId, productId, targetPrice, isActive, createdAt	Liga um usuario a um produto e define o preco alvo monitorado.
PriceHistory	id, productId, price, createdAt	Registra precos antigos quando um produto muda de preco.

Os @map indicam nomes reais das colunas no MySQL. Exemplo: passwordHash vira password_hash. Isso permite que o codigo use nomes TypeScript claros sem exigir que o banco tenha exatamente os mesmos nomes.

6. Organizacao de rotas

src/routes/index.ts junta todas as rotas principais e define os prefixos.

Prefixo	Arquivo	Responsabilidade
/auth	auth.routes.ts	Cadastro e login.
/products	product.routes.ts	CRUD, busca, ofertas, trending e historico.
/external-products	external-products.routes.ts	Busca/importacao de produtos externos.
/alerts	alert.routes.ts	Alertas autenticados do usuario.
/notifications	notification.routes.ts	Notificacoes autenticadas derivadas dos alertas.

7. DTOs e validacao

DTO significa Data Transfer Object. No projeto, os DTOs dizem qual formato a rota aceita. Eles protegem a API contra dados invalidos antes de chegar no service.

DTO	Validacoes principais
CreateUserDto	name obrigatorio, email valido, password com tamanho minimo.
CreateAlertDto	productId inteiro positivo e targetPrice numero positivo.
UpdateAlertDto	targetPrice opcional positivo; isActive opcional booleano.
ProductSearchDto	q/store/category opcionais; minPrice/maxPrice numericos; limit entre 1 e 50.
ExternalProductSearchDto	q obrigatorio; limit entre 1 e 20.
ImportExternalProductsDto	q obrigatorio, category opcional e limit controlado.
IdParamDto	id precisa ser inteiro positivo.

validation.middleware.ts recebe uma classe DTO e uma fonte: body, query ou params. Ele transforma req[source] em instancia da classe, valida com class-validator e, se passar, substitui req[source] pelo objeto validado.

```
const instance = plainToInstance(dtoClass, req[source]);
const errors = await validate(instance, { whitelist: true });
if (errors.length > 0) return next(new AppError(errorMessages, 400));
```

8. Autenticacao

A autenticacao usa senha com hash bcrypt e token JWT. O app recebe o token no login/cadastro e envia Authorization: Bearer TOKEN nas rotas protegidas.

AuthService.register

- Recebe name, email e password.
- Valida se campos existem.
- Procura email existente com prisma.user.findUnique.
- Se email ja existir, lanca AppError 409.
- Cria hash da senha com hashPassword.
- Salva usuario no banco.
- Gera token JWT com generateToken(user.id).
- Retorna user sem passwordHash e o token.

AuthService.login

- Recebe login e password.
- Permite entrar por email ou nome usando OR no Prisma.
- Se nao achar usuario, retorna credenciais invalidas.
- Compara senha digitada com passwordHash usando bcrypt.compare.
- Se senha estiver certa, gera novo token JWT.
- Retorna user e token para o Flutter salvar em AppSession.

auth.middleware.ts

```
const authHeader = req.headers.authorization;
if (!authHeader || !authHeader.startsWith("Bearer ")) {
  return next(new AppError("Token not provided", 401));
}
const payload = verifyToken(token) as AuthTokenPayload;
req.user = { id: payload.userId, email: "" };
```

Esse middleware protege /alerts e /notifications. Ele pega o token do header, verifica se e valido e coloca o id do usuario em req.user para controllers e services saberem de quem sao os dados.

9. Produtos - ProductService

ProductService concentra as regras de consulta e transformacao de produtos. Ele nao lida com HTTP diretamente; quem faz isso e ProductController. Essa separacao facilita explicar e testar regras de negocio.

Metodo	O que faz
calcularVariation	Calcula percentual de variacao entre preco atual e anterior.
normalizeProductKey	Cria chave name + category para agrupar ofertas do mesmo produto.
groupProductsByCatalog	Agrupar produtos equivalentes para comparar lojas.
getCheapestProduct	Escolhe a oferta mais barata de um grupo.
toProductSummary	Monta resumo com menor preco, maior preco, media, lojas e variacao.
buildSearchWhere	Monta filtro Prisma dinamico com q, store, category, minPrice e maxPrice.
list	Lista produtos com filtros simples.
search	Busca produtos, agrupa por catalogo e retorna os mais baratos.
trending	Calcula destaque com base em variacao e atualizacao.

Metodo	O que faz
findOffers	Mostra ofertas relacionadas do mesmo produto em lojas diferentes.
findPriceHistory	Busca historico dos produtos relacionados nos ultimos dias.
recordPriceSnapshot	Registra preco antigo e atualiza preco atual quando muda.
create/update/delete	CRUD basico de produtos.

10. Produtos externos

A API possui uma camada de providers para buscar produtos fora do banco. ExternalProductService escolhe o provider principal pelo .env. Se o Mercado Livre falhar ou nao retornar produtos, usa DummyJsonProvider como fallback para manter a demonstracao funcionando.

- ExternalProductProvider define o contrato que qualquer provedor externo deve seguir.
- MercadoLivreProvider consulta a API do Mercado Livre.
- DummyJsonProvider funciona como fonte alternativa de dados.
- importProducts busca produtos externos e salva/atualiza no MySQL.
- upsertExternalProduct cria se nao existir, atualiza se existir, e registra historico se o preco mudou.
- externalId e unico para evitar duplicar o mesmo produto externo.

Essa escolha e importante para a apresentacao: mesmo que uma API externa esteja instavel, o sistema continua demonstravel com fallback.

11. Alertas

Alertas sao o centro do aplicativo. Eles conectam usuario, produto e preco alvo.

Rota	Controller	Service	O que acontece
GET /alerts	alertController.list	alertService.list	Busca alertas ativos do usuario logado.
POST /alerts	alertController.create	alertService.create	Valida produto, evita duplicado e cria monitoramento.
PUT /alerts/:id	alertController.update	alertService.update	Confere dono do alerta e atualiza targetPrice/isActive.
DELETE /alerts/:id	alertController.delete	alertService.delete	Confere dono do alerta e remove do banco.

alertService.create explicado

- Recebe userId vindo do token e data vindo do CreateAlertDto.
- Procura produto pelo productId.
- Se produto nao existe, retorna 404 Product not found.
- Procura alerta ativo do mesmo usuario para o mesmo produto.
- Se ja existir, retorna 409 Alert already exists for this product.
- Cria alerta com userId, productId e targetPrice.
- Inclui product na resposta para o Flutter conseguir montar ProductModel.fromAlertJson.

12. Notificacoes

As notificacoes foram implementadas sem criar tabela nova. Elas sao geradas a partir dos alertas ativos. Isso foi escolhido para entregar uma funcionalidade real rapidamente, sem mudar o banco e sem criar migracao nova.

```
const reachedTarget = alert.product.currentPrice <= alert.targetPrice;
type: reachedTarget ? "priceDrop" : "alert";
```

- GET /notifications e rota autenticada.
- NotificationController pega userId do req.user.
- NotificationService busca alertas ativos com produto incluso.
- toNotification decide se o produto atingiu o alvo ou esta apenas sendo monitorado.
- getDateGroup retorna HOJE, ONTEM ou data formatada.
- getTimeAgo gera texto como 4 min atras ou 13h atras.
- Tambem cria notificacoes de sistema do tipo Alerta criado com sucesso.

Limite consciente: como nao ha tabela Notification, marcar como lida e apagar notificacao ainda fica local no app. Para evoluir, o proximo passo seria criar uma tabela notifications com isRead persistido.

13. Tratamento de erros

Arquivo	Funcao
AppError.ts	Erro personalizado com message e statusCode.
errorHandler.ts	Transforma AppError em JSON padronizado { success: false, message }.
Controllers	Usam try/catch e next(error) para mandar erro ao errorHandler.
Flutter ApiClient	Le message do JSON e cria ApiException para a tela exibir.

Essa cadeia e importante: service lanca AppError, controller repassa, errorHandler responde JSON, Flutter transforma em ApiException, tela mostra mensagem.

14. Fluxos completos para explicar

Fluxo de login

```
Flutter LoginForm
-> ApiService.login
-> POST /auth/login
-> AuthController.login
-> AuthService.login
-> Prisma user.findFirst
-> bcrypt.compare
-> JWT
-> AppSession.setAuthenticatedUser
```

Fluxo de busca/importacao

```
SearchScreen
-> GET /products/search
-> se vazio: POST /external-products/import
-> ExternalProductService busca provider externo
-> Prisma cria/atualiza produtos
-> app consulta /products/search novamente
```

Fluxo de alerta

```
NewAlertScreen
-> valida targetPrice
-> POST /alerts com Bearer token
-> authMiddleware identifica userId
-> validateDto valida productId e targetPrice
-> AlertService cria alerta
-> HomeScreen recarrega GET /alerts
```

Fluxo de notificacao

```
NotificationsScreen
-> ApiService.listNotifications
-> GET /notifications com Bearer token
-> NotificationService lista alertas do usuario
-> transforma alertas em notificacoes
-> Flutter agrupa por dateGroup e filtra por tipo
```

15. Rotas para demonstrar no Postman/Insomnia

Metodo	URL	Body/Observacao
POST	/auth/login	{"login":"admin@admin.com","password":"admin123"}
GET	/products/search?q=iphone&limit=10	Busca produtos salvos.
POST	/external-products/import	{"q":"iphone","limit":5}
GET	/alerts	Precisa Authorization: Bearer TOKEN.
POST	/alerts	{"productId":1,"targetPrice":15} com token.
GET	/notifications	Precisa Authorization: Bearer TOKEN.
GET	/products/1/offers	Mostra ofertas relacionadas.
GET	/products/1/price-history?days=90	Mostra historico de preco.

16. Decisoes tecnicas para defender

- API separada do app: melhora organizacao e permite o Flutter consumir dados reais.
- Prisma + MySQL: banco estruturado, relacoes claras e persistencia real.
- DTO + validation middleware: entrada da API validada antes da regra de negocio.
- JWT: rotas de alertas e notificacoes pertencem ao usuario logado.
- Services: regras de negocio ficam fora dos controllers.
- Providers externos: facilita trocar fonte de dados no futuro.
- Fallback DummyJson: deixa a demo funcionar mesmo se provedor principal falhar.
- Notificacoes derivadas dos alertas: solucao simples e funcional sem alterar schema agora.

17. O que ainda pode evoluir

- Criar tabela notifications para persistir lido/removido.
- Criar job agendado para atualizar precos periodicamente.
- Enviar push notification real pelo Firebase Cloud Messaging.
- Criar testes automatizados de backend.
- Melhorar validacao do LoginDto com class-validator.
- Trocar CORS '*' por origem especifica em producao.
- Corrigir textos acentuados que aparecem quebrados em alguns arquivos do app.

18. Roteiro rapido para apresentacao

- Explique que o backend e a fonte da verdade do sistema.
- Mostre schema.prisma para provar banco estruturado.
- Mostre app.ts/server.ts para explicar como a API sobe.
- Mostre routes/index.ts para explicar os modulos.
- Demonstre login e diga que gera JWT.
- Mostre authMiddleware protegendo alertas e notificacoes.
- Explique ProductService e ExternalProductService como busca/importacao.
- Explique AlertService como regra principal do projeto.
- Explique NotificationService como transformacao de alertas em notificacoes.

- Finalize dizendo que o Flutter consome tudo por ApiService.

19. Guia arquivo por arquivo - API

Arquivo	Explicacao detalhada
src/server.ts	Liga o servidor na porta definida por env.PORT.
src/app.ts	Configura middlewares globais: CORS, JSON, rotas e tratamento de erro.
src/routes/index.ts	Agrupar os modulos /auth, /products, /external-products, /alerts e /notifications.
src/config/env.ts	Carrega .env e define valores usados pelo sistema: porta, banco, JWT e provedor externo.
src/config/cors.ts	Permite chamadas HTTP vindas do app Flutter.
src/prisma/client.ts	Exporta PrismaClient unico para acesso ao MySQL.
src/types/express.d.ts	Estende Request do Express para aceitar req.user preenchido pelo authMiddleware.
src/errors/AppError.ts	Erro personalizado com status HTTP.
src/errors/errorHandler.ts	Padroniza resposta de erro para o app.
src/middleware/auth.middleware.ts	Valida token JWT e coloca userId em req.user.
src/middleware/validation.middleware.ts	Transforma e valida body/query/params usando DTOs.
src/utis/bcrypt.ts	Hash e comparacao de senha.
src/utis/jwt.ts	Geracao e verificacao de token JWT.

20. Guia de controllers e services

Camada	Arquivo	Papel
Controller	auth.controller.ts	Recebe requisicoes de cadastro/login e delega para AuthService.
Service	auth.service.ts	Regra de usuario: criar, verificar duplicado, validar senha, gerar token.
Controller	products.controller.ts	Recebe rotas de produtos e chama ProductService.
Service	product.service.ts	Busca, CRUD, agrupamento, ofertas, trending e historico de preco.
Controller	external-products.controller.ts	Recebe busca/importacao externa.
Service	external-product.service.ts	Consulta provider externo, salva/atualiza produtos e registra historico.
Controller	alert.controller.ts	Recebe listar/criar/editar/remover alertas do usuario.
Service	alert.service.ts	Valida produto, evita alerta duplicado, confere dono e persiste alertas.
Controller	notification.controller.ts	Recebe GET /notifications e retorna notificacoes do usuario.
Service	notification.service.ts	Transforma alertas ativos em notificacoes exibidas no Flutter.

21. Guia de providers externos

Arquivo	Explicacao
external-product.provider.ts	Define a interface/contrato de um provedor externo: o que precisa retornar para a API conseguir importar.
mercado-livre.provider.ts	Implementacao para consultar o Mercado Livre usando site/base URL configurados no env.
dummy-json.provider.ts	Provider alternativo usado quando o principal falha ou nao retorna produtos.
external-product.service.ts	Nao depende diretamente de um unico provedor; escolhe provider principal e fallback.

22. Contrato completo das rotas principais

Rota	Autenticada?	Entrada	Saída esperada
POST /auth/register	Nao	name, email, password	user sem senha e token JWT.
POST /auth/login	Nao	login, password	user sem senha e token JWT.
GET /products/search	Nao	q, category, store, minPrice, maxPrice, limit	Lista de ProductSummary.
GET /products/trending	Nao	limit	Produtos ordenados por variacao/atualizacao.
GET /products/:id/offers	Nao	id	Produto resumido e ofertas por loja.
GET /products/:id/price-history	Nao	id, days	Pontos historicos de preco.
POST /external-products/import	Nao	q, category, limit	Resumo de importacao e produtos criados/atualizados.
GET /alerts	Sim	Bearer token	Alertas ativos do usuario com product incluso.
POST /alerts	Sim	productId, targetPrice	Alerta criado com product incluso.
PUT /alerts/:id	Sim	targetPrice/isActive	Alerta atualizado.
DELETE /alerts/:id	Sim	id	204 sem corpo.
GET /notifications	Sim	Bearer token	Notificacoes geradas dos alertas.

23. Erros e codigos HTTP usados

Codigo	Quando aparece	Exemplo
200	Consulta ou atualizacao bem sucedida.	Login, listagem de produtos, notificacoes.
201	Criacao bem sucedida.	Cadastro, criar produto, importar produtos, criar alerta.
204	Remocao sem corpo.	DELETE /alerts/:id.
400	Validacao de DTO falhou.	productId nao inteiro, targetPrice negativo.
401	Sem token ou credenciais invalidas.	Token not provided, Invalid credentials.
403	Usuario tentando mexer em alerta de outro usuario.	Forbidden.
404	Produto ou alerta nao encontrado.	Product not found, Alert not found.
409	Conflito de regra de negocio.	Email ja registrado, alerta duplicado.
500	Erro inesperado.	errorHandler retorna Internal server error.

24. Perguntas provaveis e respostas curtas

- Por que API propria? Para o app nao depender diretamente de fontes externas e para centralizar banco, auth e regras.
- Por que MySQL? Atende dados estruturados com relacoes entre usuarios, produtos, alertas e historico.
- Por que Prisma? Evita SQL manual repetitivo e deixa models/tabelas tipados no TypeScript.
- Por que DTO? Para validar entrada antes de gravar no banco.
- Por que JWT? Para identificar o usuario em rotas protegidas sem guardar sessao no servidor.
- Por que alertas evitam duplicado? Para nao monitorar o mesmo produto duas vezes para o mesmo usuario.
- Por que notificacoes derivadas? Porque aproveita alertas ativos e entrega funcionalidade real sem nova tabela.
- O que seria proximo passo? Job de atualizacao periodica de preco, tabela notifications e push real.